UNIVERSITY OF PISA
Department of Physics

Quantum Machine Learning Exam
Academic Year 2025/2026

Comparison of Quantum Encodings for QSVM

Students:
Damiano Moscardini
Davide Petillo

Instructors:
Prof. Giuseppe Clemente
Prof. Roberto Cappuccio

July 2026

1 Introduction

This work investigates the use of a Quantum Support Vector Machine (QSVM) as a supervised classification tool based on quantum kernels.

The core idea of a QSVM is to replace the classical kernel calculation (e.g., RBF, polynomial, linear) with a Gram matrix constructed from a quantum state $|\varphi(x)\rangle$ obtained by encoding the feature vector x into a qubit register. The quantum kernel is defined as:

$$K(x_i, x_j) = |\langle \varphi(x_i) \mid \varphi(x_j) \rangle|^2$$

This is then used as a precomputed kernel within a classical SVM (scikit-learn's `SVC(kernel="precomputed")`), which handles the optimization of the separating hyperplane.

The goal of this project is not to demonstrate a strict quantum advantage, but rather to systematically compare different encoding strategies (Amplitude, Angle, Basis, IQP, Projected Quantum Kernel), including the various configurations of their quantum hyperparameters, to understand:

- which encodings can successfully extract useful non-linear relationships from the data;
- how the choice of encoding and its hyperparameters interacts with the geometric nature of the dataset;
- how these configurations perform compared to a classical baseline (RBF kernel).

To ensure a fair comparison, a brief model selection of the downstream classical SVM (a grid search on the regularization parameter C) is performed for each encoding and its respective quantum hyperparameter combinations, thereby isolating the encoding's contribution from the mere SVM optimization.

The main script, `main.py`, contains the `qsvm` function that orchestrates the model construction by relying on several auxiliary modules. Among these, the various `kernel_<encoding_name>.py` files allow for the seamless integration of different data embeddings, making the architecture highly versatile. The entire workflow is managed by the main notebook `encoding_benchmark.ipynb`, which acts as a control panel to easily configure experiments and select datasets.

2 Evaluated Encodings

Each encoding transforms the feature vector $x \in \mathbb{R}^n$, obtained after standardization and dimensionality reduction via PCA, into a quantum state on a specific number of qubits. The required number of qubits and the way features are mapped to circuit parameters vary significantly from encoding to encoding, and it is precisely this variety that is the subject of this comparison.

2.1 Amplitude Encoding

In Amplitude Encoding, the feature vector is directly encoded into the amplitudes of a normalized quantum state:

$$|\varphi(x)\rangle = \sum_{i=1}^{2^n} \frac{x_i}{\|x\|_2} \, |i\rangle$$

To represent a valid amplitude distribution, the vector must have a unit norm: in the pipeline, this is achieved natively by `qml.AmplitudeEmbedding`, via its `normalize=True` argument. The major theoretical advantage of this encoding is its qubit efficiency: with n qubits, up to 2^n real components can be encoded, making it the most "compact" among the considered encodings.

2.2 Angle Encoding

Angle Encoding maps each feature to the rotation angle of a single-qubit gate (typically R_y), applied to a dedicated qubit for each feature:

$$|\varphi(x)\rangle = \bigotimes_{i=1}^{n} R_y(x_i) \, |0\rangle$$

Since rotation angles must remain within a limited range to prevent unwanted periodic behaviors, features are rescaled using `MinMaxScaler` into the $[0.05\pi, 0.95\pi]$ interval, slightly narrower than the full $[0, \pi]$ range: this margin reduces how often validation/test features — rescaled with the same scaler fitted on the training set only — fall outside $[0, \pi]$ and require clipping. Values that still fall outside are subsequently clipped back into $[0, \pi]$. This encoding requires a number of qubits equal to the number of features (n), making it linear in complexity but highly interpretable.

2.3 Basis Encoding

Basis Encoding represents each feature as a classical bit string, mapped directly onto computational basis states $|0\rangle/|1\rangle$:

$$|\varphi(x)\rangle = |b_1 b_2 ... b_N\rangle, \quad b_i \in \{0, 1\}$$

Before discretization, features are rescaled with `MinMaxScaler` into the $[0.05, 0.95]$ range (slightly narrower than $[0, 1]$, for the same margin reasons discussed for Angle Encoding), since the `binary_conversion` function requires inputs in $[0, 1]$. Each rescaled feature is then discretized into a τ-bit binary representation (parameter `tau_bit_feature`) via `binary_conversion`. This is the only encoding considered that has a "native" and non-geometric quantum hyperparameter: increasing τ increases the resolution with which each feature is represented, at the cost of requiring $n \times \tau$ qubits, a number that grows rapidly as the required precision increases.

2.4 IQP Encoding

The Instantaneous Quantum Polynomial (IQP) Encoding employs a circuit with alternating layers of diagonal gates (typically R_z and R_{zz}), introducing correlations between pairs of features according to a specific connectivity pattern:

$$|\varphi(x)\rangle = U_{\text{IQP}}(x)H^{\otimes n}\ |0\rangle^{\otimes n}$$

Features are rescaled with `MinMaxScaler` into the $[-0.95, 0.95]$ range, slightly narrower than $[-1, 1]$ for the same margin reasons discussed for Angle Encoding, to avoid excessive phase wrapping in two-qubit gates. The main quantum hyperparameters are:

- `number_repeats`: how many times the encoding block is sequentially repeated, thereby increasing circuit depth;
- `pattern`: the topology of the two-qubit interactions, such as a nearest-neighbor chain (`chain_pattern`) or all possible pairs (`all_to_all_pattern`, yielding $\binom{n}{2}$ interactions).

This encoding features the largest search space, as it combines multiple independent quantum hyperparameters.

2.5 Projected Quantum Kernel

Unlike previous encodings, the Projected Quantum Kernel does not compute a standard kernel between state pairs $|\varphi(x_i)\rangle$ and $|\varphi(x_j)\rangle$. Instead, it projects each state onto a set of local observables: the expectation values of the Pauli matrices X, Y, and Z on each qubit, essentially retrieving the Bloch vector of each qubit. Consequently, the circuit is executed only once per sample (not for every pair), returning a vector of classical features:

$$\Phi(x) = (\langle X_0\rangle, \langle Y_0\rangle, \langle Z_0\rangle, ..., \langle X_{k-1}\rangle, \langle Y_{k-1}\rangle, \langle Z_{k-1}\rangle)$$

The final kernel is a standard classical RBF kernel computed on these vectors, with the γ hyperparameter normalized by the number of qubits. This approach circumvents the kernel concentration phenomenon, which is typical of standard kernels: as the number of qubits grows, almost all values in the Gram matrix tend to collapse towards the same number, rendering the kernel numerically indistinguishable from noise. To construct the embedding $|\varphi(x)\rangle = U(x)\ |0\rangle^{\otimes n}$, the procedure is as follows:

$$U(x) = \prod_{l=1}^{L}(W \cdot S(x))$$

where $S(x)$ denotes the local encoding block, defined as the tensor product of parametric rotations around the y-axis, depending on the individual features of the input vector x:

$$S(x) = \bigotimes_{i=0}^{n-1} R_y(x_i)$$

As in Angle Encoding, features are rescaled with `MinMaxScaler` into the $[0.05\pi, 0.95\pi]$ range before being used as rotation angles x_i. W represents the entanglement operator, typically implemented via a ring topology of CNOT gates between adjacent qubits:

$$W = \prod_{i=0}^{n-1} \text{CNOT}(i, (i+1) \bmod n)$$

The implementation of data re-uploading ($L > 1$) enriches the frequency spectrum of the model. Note that applying the operator W ensures the non-separability of the state $|\varphi(x)\rangle$.

2.6 Classical Baseline: RBF Kernel

As a non-quantum benchmark, the standard Radial Basis Function (RBF) kernel is employed, computed on features standardized via `StandardScaler`:

$$K(x_i, x_j) = \exp\left(-\gamma \, \|x_i - x_j\|^2\right)$$

The RBF baseline requires no quantum encoding and serves both as a direct accuracy reference and as a geometric comparative benchmark for the geometric coefficient described in the following section.

3 Metrics Used

To assess the quality of an encoding, the evaluation was not limited to the final accuracy of the SVM. Geometric metrics were also integrated; these can be computed directly from the Gram matrix and provide an a priori indication of kernel quality, independent of SVM training.

3.1 Kernel-Target Alignment (KTA)

Kernel-Target Alignment measures how well the Gram matrix K computed on the kernel (quantum or classical) is "aligned" with the ideal kernel $yy^\top$, constructed from the target labels $y \in \{-1, +1\}^m$:

$$\mathrm{KTA}(K, y) = \frac{\langle K, yy^\top \rangle_F}{\|K\|_F \quad \|yy^\top\|_F}$$

where $\langle \cdot, \cdot \rangle_F$ denotes the Frobenius inner product. A KTA value close to 1 indicates that the kernel already separates the two classes well geometrically, while values close to 0 signal a poor correlation between the kernel structure and the labels. This metric is used as a comparative indicator across different encodings.

3.2 Geometric Coefficient

The geometric coefficient is the geometric difference of Huang et al. (2021, "Power of data in quantum machine learning"), which compares two Gram matrices not through a direct similarity, but by asking whether the classical kernel's structure could be captured within the quantum kernel's induced function space. Given the reference classical kernel K_{RBF} and the quantum kernel K_Q (regularized as $K_Q + \lambda I$ with $\lambda = 10^{-6}$ for numerical stability):

$$\mathrm{GC}\big(K_{\mathrm{RBF}}, K_Q\big) = \sqrt{\lambda_{\max}\Big(\sqrt{K_{\mathrm{RBF}}}\, K_Q^{-1}\, \sqrt{K_{\mathrm{RBF}}}\Big)}$$

where $\lambda_{\max}$ denotes the largest eigenvalue (the matrices involved are symmetric, so this coincides with the spectral norm) and $\sqrt{K_{\mathrm{RBF}}}$ is the matrix square root of K_{RBF}. Unlike the KTA, this metric does not measure kernel quality per se, but rather how structurally different the quantum kernel is from an established classical kernel. A low geometric coefficient means the classical kernel is already well-represented within the quantum kernel's function space — no separation is possible, so the quantum encoding is unlikely to offer any advantage over the classical baseline. A high value instead means the two kernels induce genuinely different geometries, which is a **necessary** (but not sufficient) condition for a potential quantum advantage.

3.3 F1-score

The final validation metric, used to select the best model during model selection, is the F1-score, the harmonic mean of precision and recall:

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R}$$

Within the pipeline, the macro variant (`f1_score(..., average="macro")`) is computed during validation; this weighs each class equally regardless of support, proving more robust in the presence of potential class imbalances. During the final testing phase, the weighted variant is also reported to provide a more comprehensive picture of performance on the test set, alongside the full classification report (precision, recall, F1-score per class) and the confusion matrix.

4 Experiments

4.1 Qubit Limitation

A significant constraint encountered during this work is the classical simulation of quantum circuits: lacking access to real quantum hardware, every state $|\varphi(x)\rangle$ and Gram matrix is calculated by simulating the circuit's evolution on a classical computer. The cost of this simulation scales exponentially with the number of qubits n, as the associated Hilbert space has dimensionality 2^n. Both the computational time and the memory required to represent the state (or operator) scale as $O(2^n)$.

While Amplitude Encoding requires a number of qubits equal to $\lceil \log_2 n \rceil$, Angle and IQP require n, and Basis requires $n \times \tau$.

For this reason, the number of features (`number_features`) selected during PCA was deliberately kept low (on the order of ten principal components) to maintain a tractable simulation for all tested encodings, including the worst-case scenario represented by Basis Encoding with $\tau > 1$. This compromise limits the amount of original information retained from the dataset, but it is strictly necessary to conduct a thorough experimental comparison within reasonable timeframes.

4.2 Model Selection

For each encoding, and for each combination of its quantum hyperparameters, a model selection of the classical SVM is performed. This follows a two-level scheme designed to minimize the number of times the quantum kernel (the most computationally expensive operation) is recalculated:

1. Outer level (quantum hyperparameters): Training and validation sets are merged into a single cross-validation set. For every combination in the quantum hyperparameter grid (e.g., `number_repeats` and `pattern` for IQP, or `gamma` and `layer_number` for the Projected Quantum Kernel), the Gram matrix is computed only once on this merged set, as well as against the test set, using the `calculate_kernel` function provided by the corresponding encoding module.

2. Inner level (classical hyperparameters): Based on the same precomputed Gram matrix, a grid of values for the SVM regularization parameter C is explored. Each combination is not evaluated on a single train/validation split, but rather through a 5-fold Stratified Cross-Validation (`StratifiedKFold`). The indices of the 5 folds, computed once and shared across all classical combinations, are used to directly slice rows and columns of the already calculated Gram matrix without ever recomputing the quantum kernel. For each fold, the SVM is trained on the internal training block and evaluated (using the macro F1-score) on the internal validation block; the combination's score is the mean (and standard deviation) of the F1-score across the 5 folds.

The use of `StratifiedKFold` (as well as the initial dataset split into train and test via `train_test_split` with `stratify=y`) ensures that the class proportions present in the original dataset are preserved in each generated subset. This precaution is particularly crucial when the number of samples is small or when classes are imbalanced: a purely random split could inadvertently produce folds or test sets with very poor (or zero) representation of one of the

classes. Stratification guarantees that every evaluation is conducted on a truly representative sample of the original classification problem.

Once the optimal combination (both quantum and classical) with the highest average cross-validation score is identified, the final model is retrained on the entire cross-validation set (training and validation merged). This leverages all available data before the independent evaluation on the unseen test set.

Compared to a single train/validation split, this scheme reduces the variance in performance estimation for each classical hyperparameter combination. Furthermore, the standard deviation across folds provides an additional metric of stability for each configuration. The total number of evaluated combinations remains the product of the two grids (scikit-learn's `ParameterGrid`); for each, the exact hyperparameters, the mean and standard deviation of the F1-score across folds, and the distinct computation times for the kernel and SVM cross-validation are logged. The entire history of results is exported to an `.xlsx` file, automatically highlighting the row corresponding to the winning configuration to facilitate manual inspection.

This setup ensures that comparisons between different encodings (and different configurations of the same encoding) are neither biased by suboptimal SVM regularization parameter selection nor heavily dependent on a highly favorable or unfavorable single train/validation split. Each encoding is evaluated in its optimal classical configuration according to a robust estimate, effectively isolating the quantum encoding's specific contribution to the model's separating capability.

5 Data Analysis

5.1 Datasets Used

The benchmark is conducted on six datasets, carefully chosen to cover various geometries and thus stress-test different hypotheses regarding which encoding might perform best:

- **Linear**: Generated via `make_classification` (one cluster per class, no redundant features), linearly separable by design. This acts as the "easy" control case: an encoding that struggles here suffers from a baseline expressivity issue, not an inability to capture non-linearity.
- **Circles**: `make_circles`, two classes distributed on concentric circles. Not linearly separable in the original space; it favors encodings capable of inducing radial decision boundaries.
- **Moons**: `make_moons`, two interleaved half-moons.
- **Quantum_Ad_Hoc**: A 2D synthetic dataset with labels given by $\text{sign}(\sin(x_1) \cdot \cos(x_2))$, explicitly constructed to favor intrinsically periodic encodings (e.g., Angle, IQP, Projected Quantum Kernel).
- **High_Dim_XOR**: Generalized XOR in `number_features` dimensions; the label is given by the sign of the product of all feature signs. This dataset rewards encodings capable of simultaneously capturing high-order correlations across all features, rather than just pairwise interactions.
- **Breast_Cancer**: Real-world dataset from scikit-learn.

For each dataset, the analysis is conducted across three experimental scales, denoted as `number_features / number_samples`: `4/200`, `8/200`, and `16/400`. Across all three, 20% of the data is reserved for the test set from the start (stratified), while the remaining 80% is used for the aforementioned 5-fold cross-validation. The performance comparison presented here is strictly between the five quantum encodings: the RBF baseline, introduced earlier as a conceptual reference and benchmark for the Geometric Coefficient, is excluded from this direct accuracy comparison.

5.2 Methodological Note: Statistical Significance and an Apparent Artifact

Before detailing the results, two clarifications are necessary to avoid interpreting statistical noise as signal.

First, due to computational limits, dataset sizes remained consistently small (200 or 400). This results in a test set of 40 (or 80) samples. Therefore, a 3-4 percentage point difference in F1 corresponds to a mere 1-2 differently classified samples. Such minor discrepancies cannot serve as the basis for solid conclusions, especially when comparing encodings with similar performance. Consequently, performance gaps of this magnitude are highlighted as indicative but are not treated as proof of one encoding's definitive superiority over another.

Secondly, the Circles, Moons, and Quantum_Ad_Hoc datasets yield identical results between `8/200` and `4/200`. At first glance, this might appear to be a pipeline error (e.g., a duplicated file or stale cache). It is not: these three datasets are inherently two-dimensional (only two "true" features, regardless of the input dimension requested), meaning the downstream PCA cannot extract more than 2 useful components in either case. Given the same random seed and sample size (200 in both configurations), the preprocessing, splitting, and subsequent phases yield bit-for-bit identical results. Therefore, comparing `8/200` to `4/200` for these three datasets is interpretively irrelevant, as the columns align by design. The three-

scale comparison is only meaningful for Linear, High_Dim_XOR, and Breast_Cancer, where the number of features genuinely usable by PCA scales accordingly.

5.3 Comparison Between Encodings, Dataset by Dataset

The following tables report the F1-score (weighted) of the best configuration for each encoding, selected via cross-validation, at the three experimental scales (for Circles, Moons, and Quantum_Ad_Hoc, refer to the note above: columns `4/200` and `8/200` coincide). The two images provided per dataset (at 8 qubits / 200 samples) display the confusion matrix and the Gram matrix (sorted by class) for both the best and worst encodings (evaluated on the test set, after optimal hyperparameters were chosen), allowing for a direct comparison of the underlying geometric structure.

5.3.1 Linear

Encoding	F1 (4/200)	F1 (8/200)	F1 (16/400)
Amplitude	0.444	0.500	0.547
Angle	0.925	0.900	0.950
Basis	0.925	0.693	0.333
IQP	0.900	0.848	0.937
Projected	0.798	0.744	0.825

On a problem that is linearly separable by design, any reasonable encoding is expected to perform well. This holds true for Angle and IQP at all scales, but crucially not for Amplitude, which remains stalled near random chance. The root cause is structural: Amplitude Encoding normalizes every sample, thereby discarding the absolute scale information of the vector —the exact information separating the classes in this setup. Furthermore, Basis Encoding deteriorates sharply as the scale increases ($0.925 \rightarrow 0.333$): with higher bit-precision per feature (`tau_bit_feature`), the Hilbert space grows faster than the sample size (which remains unchanged from `4/200` to `8/200`) can support.

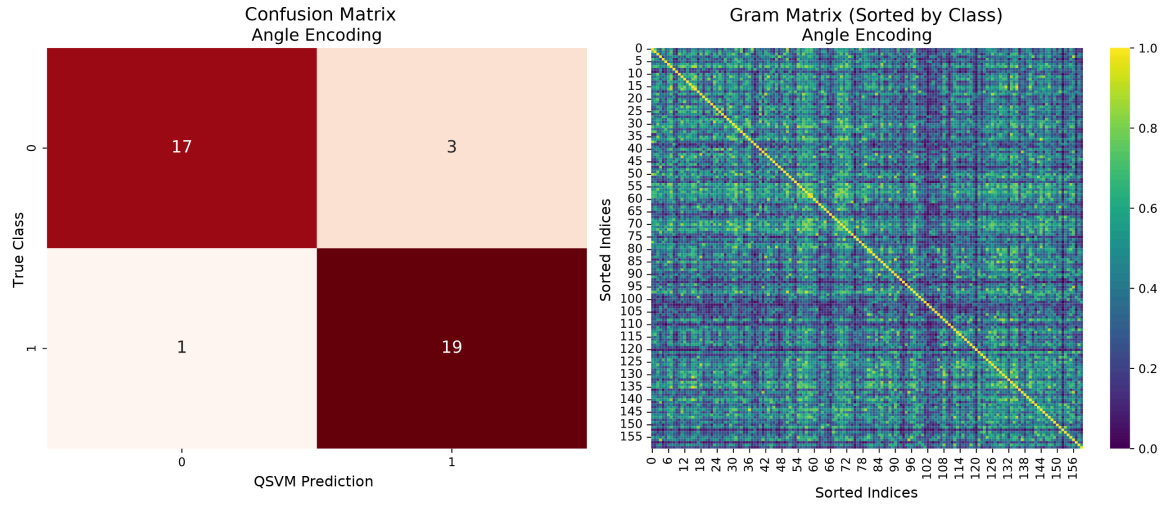

Figure 1: Linear, 8 features/200 samples, Angle Encoding, Acc = 0.900 (best).

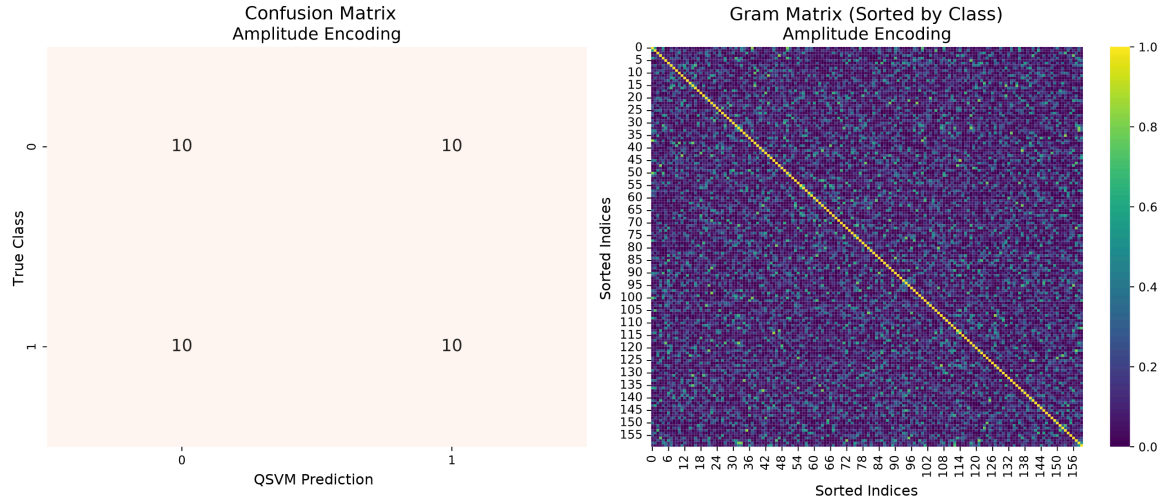

Figure 2: Linear, 8 features/200 samples, Amplitude Encoding, Acc = 0.500 (worst, chance level).

5.3.2 Circles

Encoding	F1 (4/200)	F1 (8/200)	F1 (16/400)
Amplitude	0.437	0.437	0.511
Angle	1.000	1.000	1.000
Basis	0.899	0.899	0.460
IQP	1.000	1.000	1.000
Projected	1.000	1.000	1.000

Circles is the dataset where the comparison is least informative: three out of five encodings achieve F1 = 1.000 at just 4 qubits. Given a test set of only roughly 40 samples with simple radial geometry, this points to problem saturation rather than strict equivalence among Angle, IQP, and Projected. The previous structural observations regarding Amplitude and Basis still apply here.

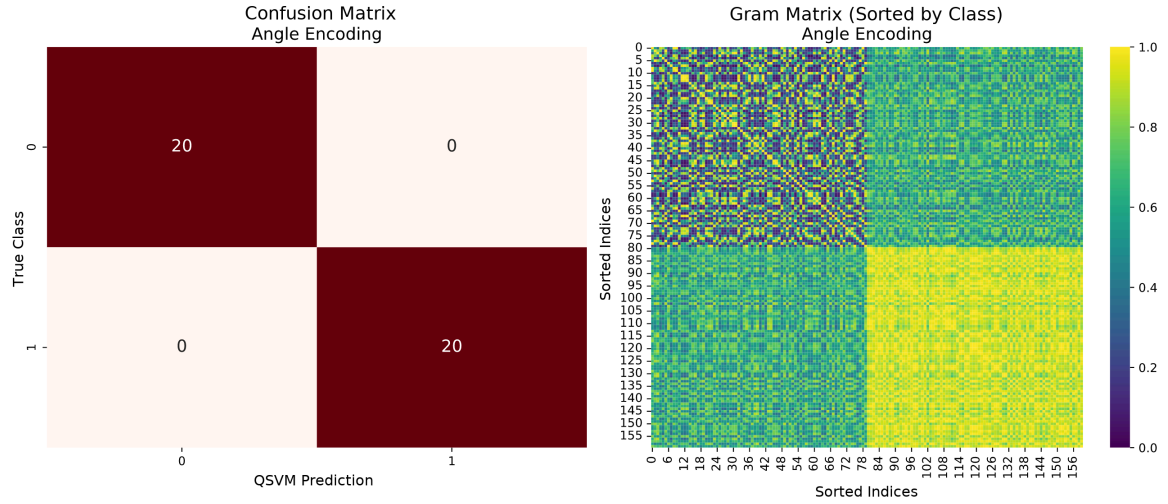

Figure 3: Circles, 8 features/200 samples, Angle Encoding, Acc = 1.000 (best).

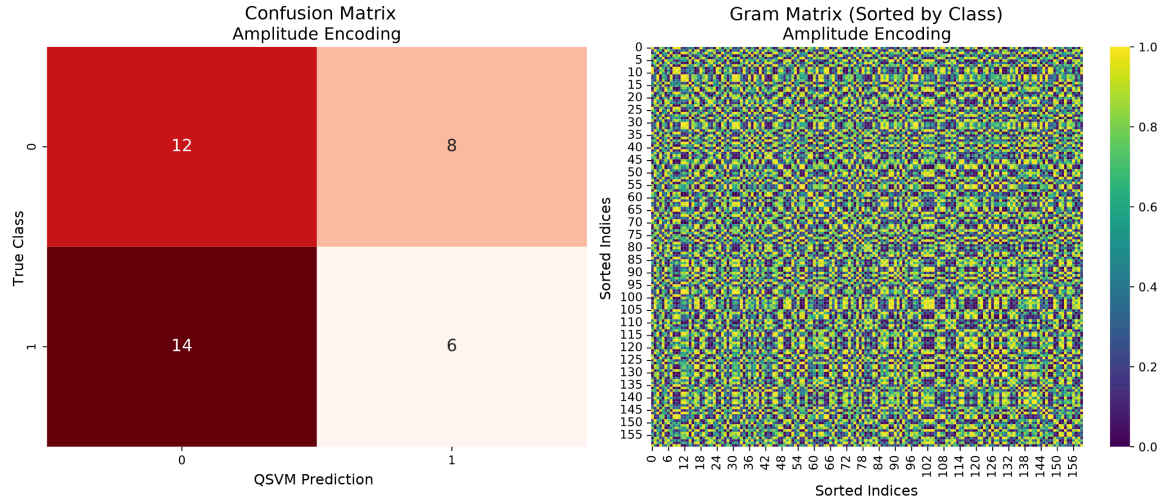

Figure 4: Circles, 8 features/200 samples, Amplitude Encoding, Acc = 0.450 (worst).

The Gram matrix for Angle displays a clear block structure (one for each class), whereas Amplitude's matrix is visually indistinguishable from noise.

5.3.3 Moons

Encoding	**F1 (4/200)**	**F1 (8/200)**	**F1 (16/400)**
Amplitude	0.437	0.437	0.449
Angle	0.975	0.975	0.963
Basis	0.899	0.899	0.875
IQP	0.975	0.975	0.988
Projected	0.975	0.975	0.988

The pattern is similar to Circles but slightly less saturated: Amplitude remains the clear loser, while the other four encodings are highly competitive and functionally equivalent (differences fall within 1-2 percentage points, well within the statistical noise threshold discussed earlier).

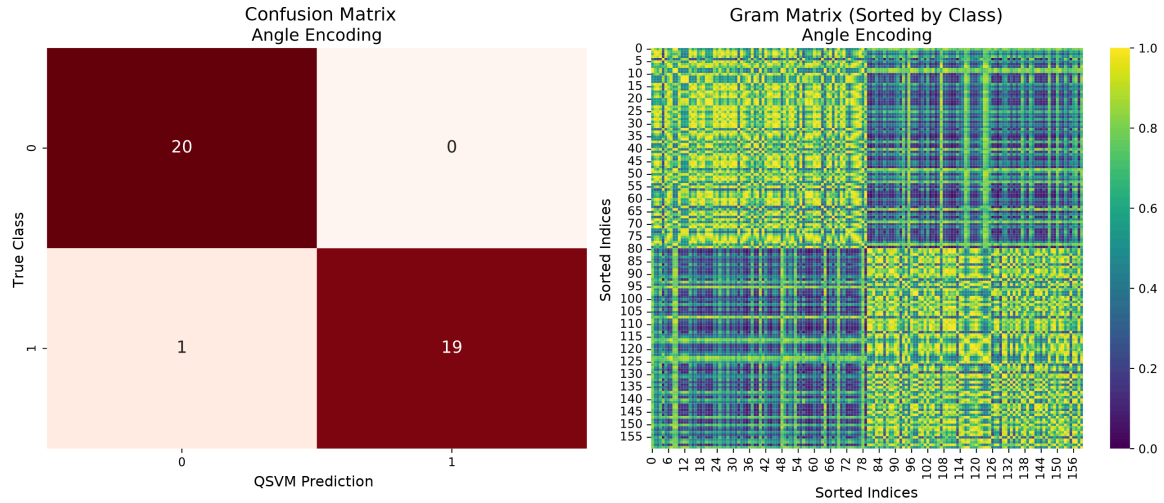

Figure 5: Moons, 8 features/200 samples, Angle Encoding, Acc = 0.975 (best, tied with IQP and Projected).

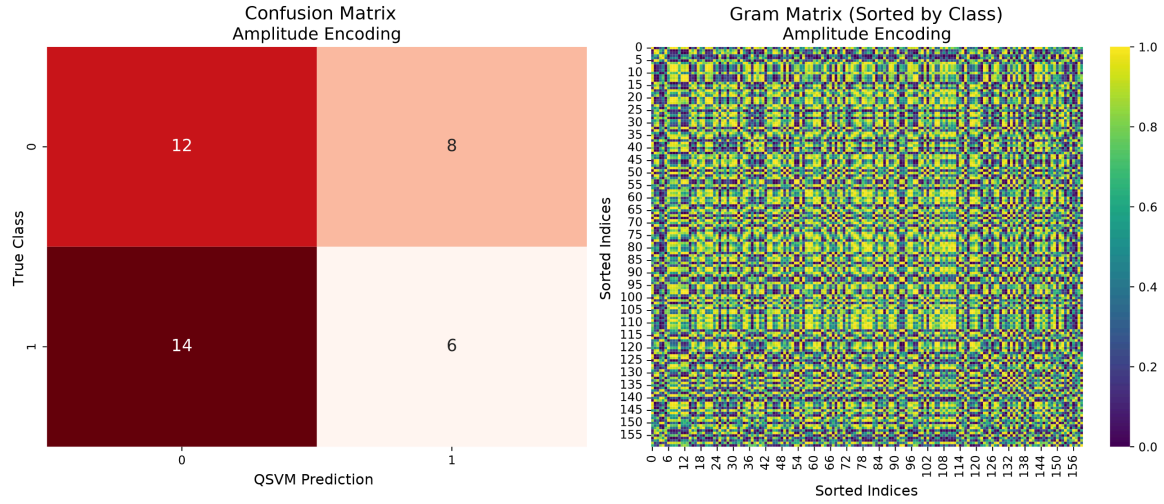

Figure 6: Moons, 8 features/200 samples, Amplitude Encoding, Acc = 0.450 (worst).

Here, the block structure for Angle is even more pronounced.

5.3.4 Quantum_Ad_Hoc

Encoding	F1 (4/200)	F1 (8/200)	F1 (16/400)
Amplitude	0.400	0.400	0.499
Angle	0.617	0.617	0.725
Basis	0.572	0.572	0.536
IQP	0.662	0.662	0.737
Projected	0.850	0.850	0.975

The dynamic shifts significantly here: no encoding achieves perfect separation at `4/200` or `8/200`, and Projected emerges as the clear winner across all scales, showing marked improvement when moving to `16/400` (0.850 → 0.975). Remarkably, although this dataset is explicitly designed to favor "rotational" encodings (Angle, IQP, Projected) via the $\sin(x_1)\cos(x_2)$ pattern, Projected leads by a wide margin, leaving Angle and IQP hovering around 0.6-0.7.

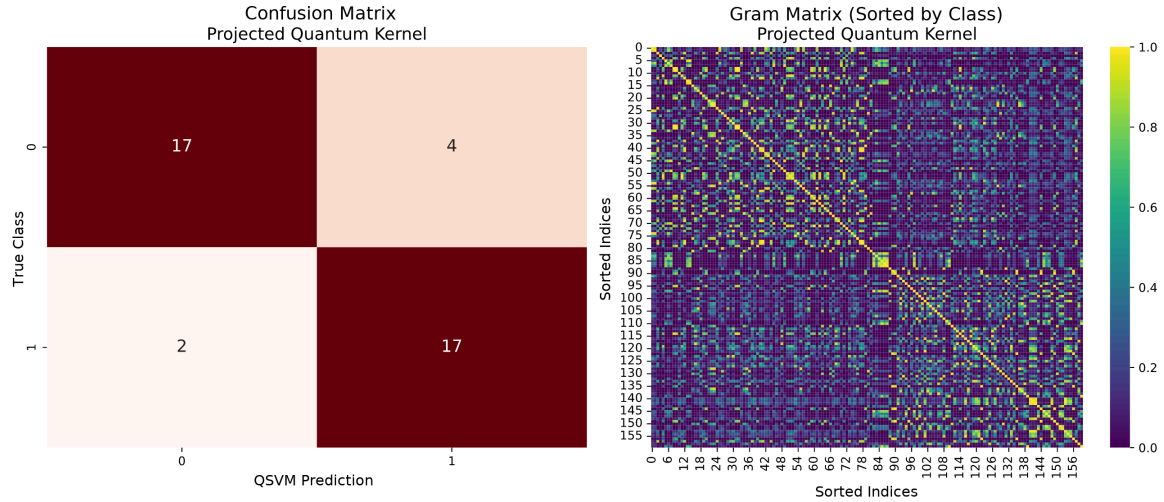

Figure 7: Quantum_Ad_Hoc, 8 features/200 samples, Projected Quantum Kernel, Acc = 0.850 (best).

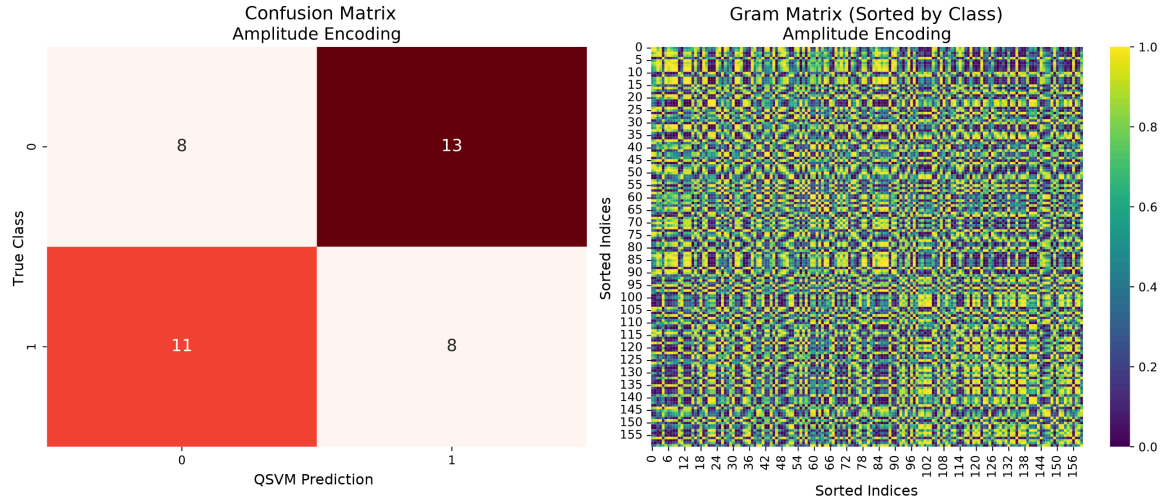

Figure 8: Quantum_Ad_Hoc, 8 features/200 samples, Amplitude Encoding, Acc = 0.400 (worst).

5.3.5 High_Dim_XOR

Encoding	F1 (4/200)	F1 (8/200)	F1 (16/400)
Amplitude	0.624	0.550	0.474
Angle	0.600	0.459	0.362
Basis	0.617	0.422	0.347
IQP	0.645	0.400	0.432
Projected	0.594	0.421	0.448

It must be stated clearly before analyzing the numbers: practically no encoding functions effectively on High_Dim_XOR. The absolute best (Amplitude at `8/200`) reaches only $F1 = 0.550$, while the rest oscillate between 0.40 and 0.65. The gaps between "best" and "worst" are entirely subsumed by statistical noise. Declaring a "winner" here would be misleading; the following comparison should be read as a contrast between two different modes of failure, rather than success versus failure.

High_Dim_XOR is also the only dataset where performance degrades universally when scaling from 4 to 16 qubits. This aligns perfectly with the nature of the problem: generalized XOR requires capturing an n-order correlation (the sign of the product of all features), not merely pairwise relationships. As the feature count (and thus qubit count) increases, the space in which this correlation resides grows exponentially while the sample size remains comparatively small. This is the exact curse of dimensionality noted earlier regarding classical simulation, now observed from a learning perspective: even when simulation is tractable, the learning problem itself becomes overwhelmingly difficult. Not even IQP with an "all-to-all" pattern (which explicitly embeds pairwise correlations across all features) can compensate.

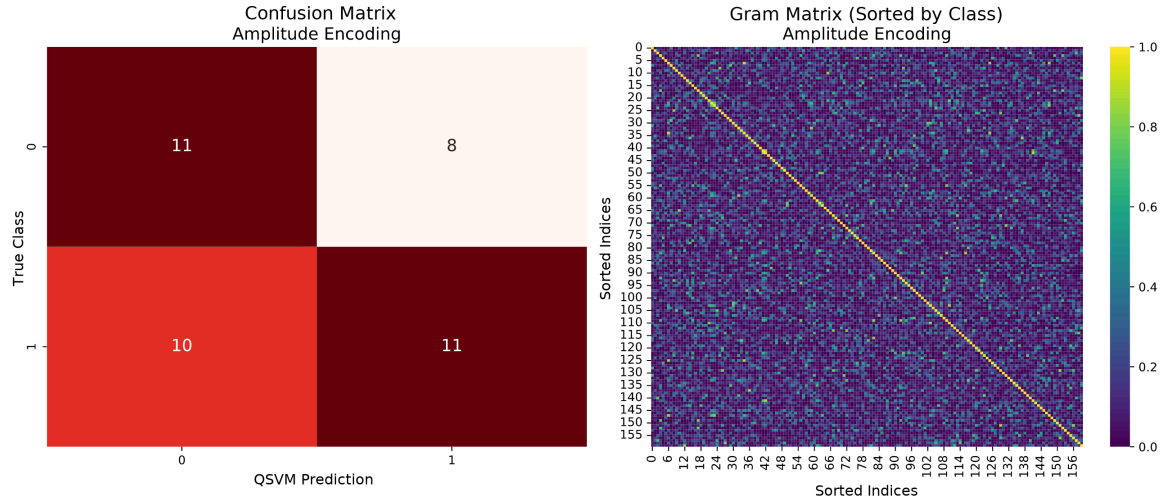

Figure 9: High_Dim_XOR, 8 features/200 samples, Amplitude Encoding, Acc = 0.550 (the "best", but hovering near chance).

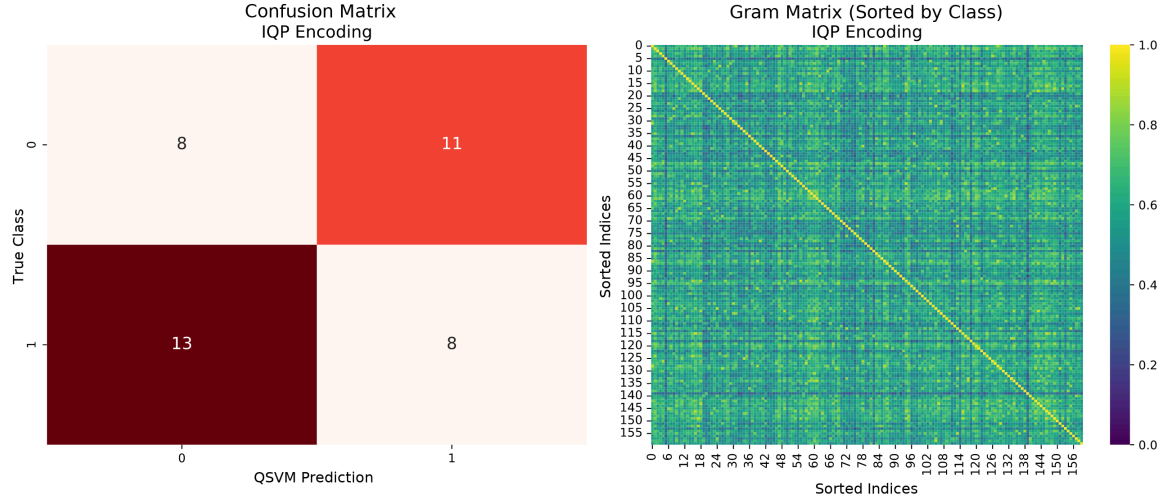

Figure 10: High_Dim_XOR, 8 features/200 samples, IQP Encoding ("all-to-all" pattern), Acc = 0.400 (the worst).

Even the Gram matrix for the top performer (Amplitude) fails to reveal any recognizable structure.

5.3.6 Breast_Cancer

Encoding	F1 (4/200)	F1 (8/200)	F1 (16/400)
Amplitude	0.567	0.673	0.794
Angle	0.949	0.949	0.937
Basis	0.732	0.688	0.488
IQP	0.975	0.924	0.962
Projected	0.975	0.975	0.938

On the real-world dataset, IQP and Projected perform strongest across all scales. Angle performs well but sits slightly behind both. Notably, Amplitude is the sole encoding whose performance improves significantly and monotonically with scaling ($0.567 \rightarrow 0.673 \rightarrow 0.794$). This is logically consistent: with more features (and thus more qubits, since Amplitude scales

logarithmically with feature count), the scalar information lost during normalization becomes proportionally less detrimental to the whole.

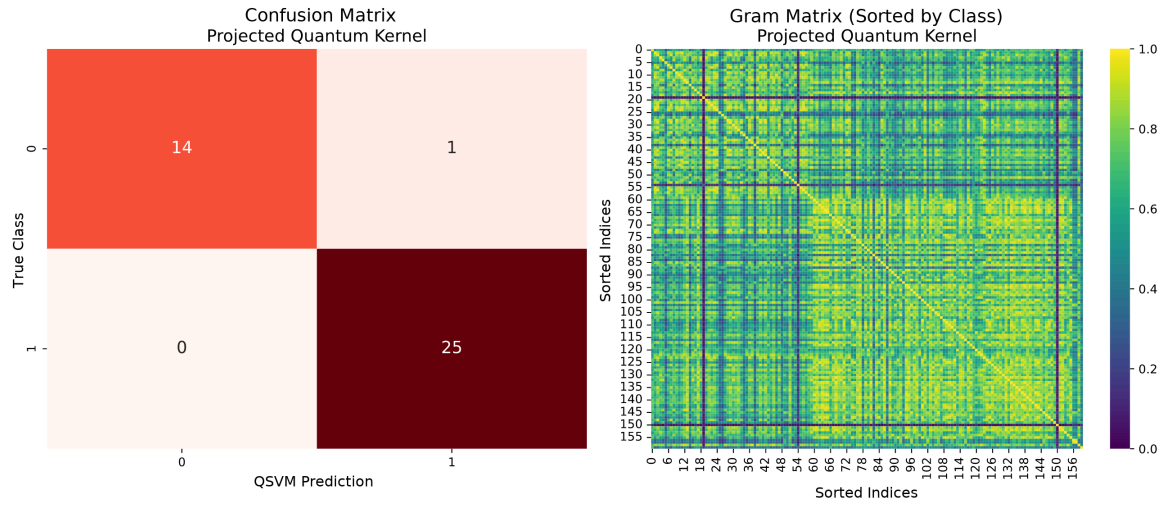

Figure 11: Breast_Cancer, 8 features/200 samples, Projected Quantum Kernel, Acc = 0.975 (best).

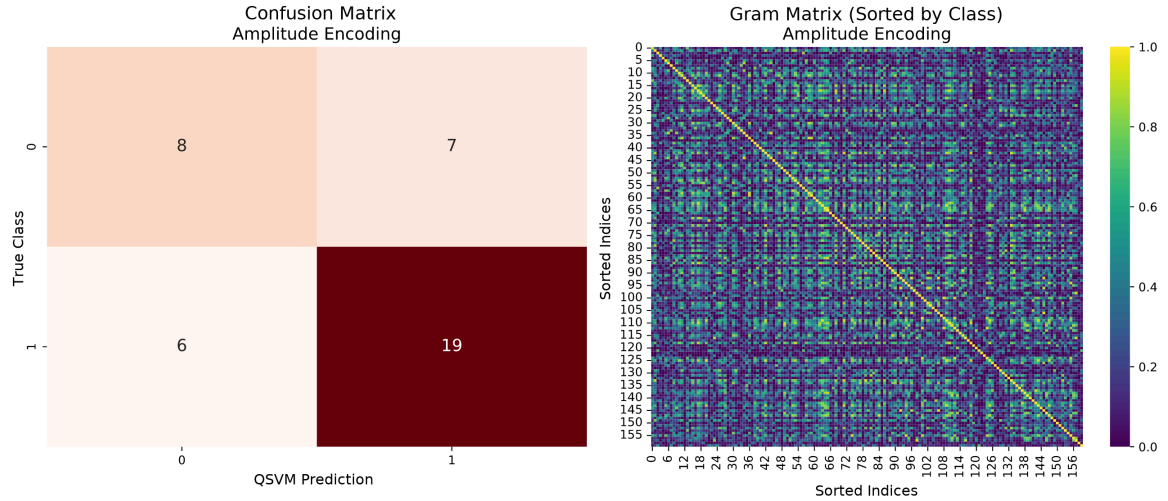

Figure 12: Breast_Cancer, 8 features/200 samples, Amplitude Encoding, Acc = 0.675 (worst).

5.4 KTA and Geometric Coefficient as Performance Predictors

Aggregating all combinations of encoding, dataset, and scale (90 rows total: 3 scales × 6 datasets × 5 encodings), the Pearson correlation between Quantum KTA (computed on training data) and test F1 is $r = 0.47$ ($p < 10^{-5}$; Spearman $\rho = 0.44$). This is a positive and non-negligible correlation, yet far from a strong bond. When broken down by encoding, the correlation shifts substantially:

Encoding	**Pearson KTA-F1** ($n = 18$)
Basis	0.79
Amplitude	0.70
Angle	0.63
IQP	0.53
Projected	0.45

KTA therefore serves as a useful proxy indicator: a severely low KTA (as seen systematically with Amplitude) almost guarantees poor performance. However, it is unreliable as a fine-tuned selection criterion between closely matched configurations. Tellingly, the correlation between training KTA and test F1 is weakest in the overall best-performing encoding (Projected). This implies KTA alone cannot confidently determine the better choice between two comparably performing quantum hyperparameter configurations within that encoding framework.

Conversely, the Geometric Coefficient against the RBF baseline shows no useful correlation with F1 ($r = -0.14$, $p = 0.20$; $\rho \approx 0$, $p = 0.94$: both statistically indistinguishable from zero). This is not a failure of the metric, but rather a reflection of what it measures: the Geometric Coefficient quantifies how structurally different the quantum kernel is from the classical RBF kernel, not whether that divergence improves classification accuracy. An encoding can "view" data in a vastly different geometric space than RBF (high GC) yet yield far worse results, as repeatedly observed with Amplitude. Geometric divergence from classical kernels does not intrinsically equate to predictive advantage.

5.5 The Winner?

Among the five encodings, the **Projected Quantum Kernel** is undeniably the most consistent: it ties or outright holds the highest F1-score across 4 of the 6 datasets at `8/200` (Moons, Quantum_Ad_Hoc, Breast_Cancer, and the saturated Circles at 1.000). This dominance largely persists across `4/200` and `16/400`. Angle and IQP remain highly competitive everywhere except on Quantum_Ad_Hoc, where Projected takes a commanding lead. On High_Dim_XOR, as previously noted, there is no true winner, merely a "least bad" option.

An essential computational cost factor, entirely absent from the pure F1 metric, must also be considered: aggregating kernel calculation times across all cross-validation combinations reveals IQP to be drastically more expensive (averaging roughly 55 times longer per combination than Projected), despite never decisively outperforming it in accuracy. At parity of predictive capability, Projected yields comparable or superior results at a fraction of the computational overhead.